

FREE BUILD GUIDE

Build Your Own TikFulfill

A complete step-by-step guide to building a TikTok Shop auto-fulfillment dashboard — using **VS Code + Claude Code**. No prior coding experience needed. Every prompt is copy-paste ready.

By Imtiaz Ahmad · AI Soft Tech Solution · aisofttechsolution.com

What's inside

1. What you're building (the full feature list)
2. What you need (free tools only)
3. Install VS Code, Node.js & Claude Code
4. Set up your project folder
5. Prompt 1 — Dashboard shell + live stats bar
6. Prompt 2 — Live order queue with dispatch countdowns
7. Prompt 3 — Multi-shop tabs (the agency superpower)
8. Prompt 4 — At-risk detection + filters
9. Prompt 5 — Bulk label printing
10. Prompt 6 — Courier assignment + pickup scheduling
11. Prompt 7 — Exception queue
12. Prompt 8 — Auto-print rule toggle
13. Connect real TikTok Shop data (API)
14. Make it yours — customization ideas
15. Put it online (free hosting)
16. Troubleshooting

1. What You're Building

TikFulfill is an **auto-fulfillment command center** for TikTok Shop sellers. TikTok gives sellers a **48-hour dispatch window** — miss it and you get punished: payouts held, FYP reach cut. When a video goes viral (500–2,000 orders in a day), manual processing collapses. This dashboard fixes that.

#	Feature	What it does
1	Live order queue	Every order appears in one table the second it's placed
2	Dispatch countdown	Each order shows live time left in its 48-hour window
3	Multi-shop tabs	Manage several client shops from one dashboard (agency-ready)
4	At-risk flagging	Orders under 4 hours left get highlighted automatically
5	Stats bar	Orders today, dispatched, at-risk, average dispatch time
6	Bulk label printing	Select orders → print all shipping labels in one click
7	Auto-print rule	Toggle: labels auto-print when an order has < 4h left
8	Courier assignment	Assign Leopards / TCS / PostEx per order, schedule pickup
9	Exception queue	Failed pickups, bad addresses, COD mismatches — with one-click resolve
10	Filters	All / Pending / At-risk views

How this guide works: you don't write code. You copy each **Prompt** box below, paste it into Claude Code inside VS Code, press Enter, and review what it builds. Each prompt builds on the last. Total build time: about 60–90 minutes.

2. What You Need

- A computer (Windows, Mac, or Linux)
- Internet connection
- About 60–90 minutes
- **Zero** coding experience — Claude Code writes the code, you direct it

Everything in this guide is **free**.

3. Install VS Code, Node.js & Claude Code

Step 3.1 — Install VS Code

1. Go to **code.visualstudio.com** and download VS Code for your system.
2. Run the installer. Accept all defaults.
3. Open VS Code once to make sure it launches.

Step 3.2 — Install Node.js

1. Go to **nodejs.org** and download the **LTS** version (v20 or newer).
2. Run the installer. Accept all defaults.
3. Verify: open a terminal (in VS Code: `Terminal` → `New Terminal`) and type `node --version` . You should see a version number like `v20.x.x` .

Step 3.3 — Install Claude Code

1. In VS Code, click the **Extensions** icon on the left sidebar (or press `Ctrl+Shift+X`).
2. Search for **"Claude Code"** (by Anthropic).
3. Click **Install**.
4. Open it from the sidebar and **sign in** with your Anthropic account (create one free at claude.ai if you don't have one).

Checkpoint: VS Code is open, the terminal shows a Node version, and Claude Code is installed and signed in. You're ready.

4. Set Up Your Project Folder

1. In VS Code: **File** → **Open Folder**, create a new folder called `tikfulfill` and open it.
2. Inside it, create one empty file: `index.html` (right-click the folder → `New File`).
3. Open Claude Code in the sidebar and make sure it's pointing at this folder.

Why one file? We're building the whole dashboard as a single `index.html` with embedded CSS and JavaScript. It runs by double-clicking it — no servers, no build tools, no complexity. You can split it into files later once you're comfortable.

5. Prompt 1 — Dashboard Shell + Live Stats Bar

Copy everything in the box below and paste it into Claude Code. Press Enter.

```
Create the foundation of a TikTok Shop fulfillment dashboard called
"TikFulfill" in index.html (single file, embedded CSS + JS).
```

Build:

1. A dark-themed header with the title "TikFulfill" (accent color cyan #22d3ee) and a pulsing "Live" indicator dot.
2. A pink warning banner that reads: "X orders are near the 48-hour dispatch window. TikTok penalizes late dispatch with suppressed payouts and FYP reach cuts." — X should be a number that updates from JS data later.
3. A stats bar with 4 cards: "Orders today", "Dispatched", "At risk", "Avg dispatch". Give each card an id (statOrders, statDisp, statRisk, statAvg) so JS can update the numbers. Use distinct accent colors: cyan, green, pink, purple.
4. Use the Inter font (Google Fonts link is fine), modern rounded cards, subtle hover lift on stat cards.

For now, fill the stats with placeholder numbers in JS (e.g. 1284, 1102, 37, 6.2h). Keep all JS in a script tag at the bottom. Make it responsive.

Verify: Open `index.html` in your browser (double-click it). You should see the dark dashboard, the pink banner, and 4 stat cards with numbers. If anything looks broken, tell Claude Code what you see and ask it to fix it.

6. Prompt 2 — Live Order Queue with Dispatch Countdowns

```
Add the core order queue to the TikFulfill dashboard in index.html.
```

1. Create a JS array of ~25 sample orders. Each order: id like "TT-90121", customer name (use Pakistani names: Ayesha Khan, Bilal Ahmed, Fatima Raza, etc.), items count (1-5), secondsLeft (random between 600 and 30000), status ("pend" or "ok"), shop id.
2. Render a table with columns: Order | Customer | Items | Dispatch in | Status. The "Dispatch in" column shows a live ticking countdown (HH:MM:SS format) that updates every second from secondsLeft. Style it in monospace cyan.
3. Status shows a pill: "Pending" (cyan) or "Dispatched" (green).
4. Above the table add filter buttons: All | Pending | At risk — clicking filters the visible rows. Give them an active state style.

5. Keep the stats bar updating: Orders today = total orders, Dispatched = count of ok, At risk = count with secondsLeft under 14400 (4 hours).

Do not remove anything already built. Only add.

Verify: Refresh the page. The table appears, countdowns tick down live every second, and the filter buttons work. The stats numbers should now reflect the sample data.

7. Prompt 3 — Multi-Shop Tabs (The Agency Superpower)

This is the feature that makes TikFulfill agency-ready: one dashboard, many client shops.

Add multi-shop support to the TikFulfill dashboard in `index.html`.

1. Add a shop tab bar above the stats: tabs for "All shops" (1284 orders), "GlowKart" (612), "DesiNest" (418), "FitFuel" (254). Style the active tab with cyan border.
2. Give each sample order a shop field (distribute across glowkart, desinest, fitfuel). Clicking a shop tab filters the order table AND recalculates the stats bar for that shop only. "All shops" shows everything.
3. The pink warning banner's order count should also update per selected shop.

Do not remove anything already built. Only add.

Verify: Click each shop tab — the table, stats, and banner count should all change. This is exactly how an agency would monitor multiple clients from one screen.

8. Prompt 4 — At-Risk Detection + Filters

Add at-risk order handling to the TikFulfill dashboard in `index.html`.

1. Any order with less than 4 hours (14400 seconds) left gets the CSS class "risk": its table row gets a subtle pink background tint and its countdown turns pink and bold.
2. Make the "At risk" filter button show only these orders.
3. Add a small toggle switch labeled "Auto-print when < 4h left" near the filters. When ON, any order that crosses under 4 hours should trigger a toast notification saying "Auto-print: label queued for TT-XXXXX". Store the toggle state in a JS variable.
4. Add a tiny toast function that shows a floating notification bottom-right for 3 seconds.

Do not remove anything already built. Only add.

Verify: Orders under 4 hours glow pink. Flip the auto-print toggle on and watch for toast notifications. The "At risk" filter isolates them.

9. Prompt 5 — Bulk Label Printing

Add bulk shipping-label printing to the TikFulfill dashboard in index.html.

1. Add a checkbox column to the order table (header checkbox selects/deselects all visible rows).
2. Below the table, add a bar that reads: "N selected — [Print M labels]" where N/M update live. The button should be prominent (cyan).
3. Clicking the button opens a modal titled "Printing labels" listing each selected order id with a fake label preview line (order id, customer, items), then a "Done" button that marks those orders as Dispatched (status pill turns green, countdown stops) and closes the modal.
4. After printing, the stats bar must update (Dispatched up, Pending down).

Do not remove anything already built. Only add.

Verify: Select a few orders, click print, watch the modal, click Done — those orders turn green and the stats update. This is the one-click workflow that replaces manual label printing.

10. Prompt 6 — Courier Assignment + Pickup Scheduling

Add courier assignment to the TikFulfill dashboard in index.html.

1. Add a "Courier" column to the order table with a dropdown per row: options "Leopards", "TCS", "PostEx". Default: auto-assign round-robin across the three.
2. Add a small "📅 Pickup" button per row. Clicking it sets that order's status pill to show "Pickup scheduled" (purple pill) via a toast confirmation "Pickup scheduled for TT-XXXXX with [courier]".
3. Selections must persist while filtering (store courier choice on the order object, not just the DOM).

Do not remove anything already built. Only add.

Verify: Change couriers on a few rows, schedule pickups, then filter — the choices should stick.

11. Prompt 7 — Exception Queue

Add an exception queue to the TikFulfill dashboard in index.html – this is where problem orders go instead of exploding in the inbox.

1. Below the main table, add a section titled "🔥 Exception queue – needs a human".
2. Seed 3 sample exceptions:
 - "Failed pickup – rider found shop closed" → detail "TT-90098 · Leopards · retry pickup"
 - "Address issue – incomplete house number" → detail "TT-90102 · TCS · confirm with customer"
 - "COD amount mismatch on label" → detail "TT-90111 · PostEx · reprint label"
3. Each exception card shows the title, the detail line, and a "Resolve" button. Clicking Resolve removes the card with a toast "Exception resolved".
4. Style the section distinctly (amber/red tint) so it visually stands apart from the normal queue.

Do not remove anything already built. Only add.

Verify: The exception section appears below the table. Clicking Resolve removes cards one by one.

12. Prompt 8 — Auto-Print Rule Toggle (Polish)

Polish the auto-print rule in the TikFulfill dashboard in index.html.

1. Make the "Auto-print when < 4h left" toggle visually clear (cyan when ON, grey when OFF) with a smooth sliding knob animation.
2. When the toggle is ON and an order crosses below 4 hours during the live countdown, automatically check its checkbox AND show the toast "Auto-print: label queued for TT-XXXXX". Don't auto-open the print modal – the user reviews and clicks Print when ready.
3. Add a footer line: "TikFulfill concept – built with VS Code + Claude Code".

Review the whole file for any JS errors in the console and fix them. Do not remove anything already built.

Verify: Open browser dev tools (F12 → Console) — there should be zero errors. Toggle the switch a few times. Everything from prompts 1–7 still works. Congratulations — the dashboard is complete!

13. Connect Real TikTok Shop Data (API)

Right now your dashboard runs on **sample data** — perfect for demos. To use it for a real shop, connect TikTok Shop's API. This is the one step that needs a developer's eye (or one more Claude Code session).

How it works — the big picture

1. **Get API access:** register at `partner.tiktokshop.com` → create an app → get your `app_key` and `app_secret`. The shop owner authorizes your app once.
2. **Pull orders:** TikTok's `/order/search` endpoint returns new orders. Each order has a `create_time` — your 48-hour countdown is `create_time + 48h - now`.
3. **Swap the data source:** in your `index.html`, replace the sample `0` array with a `fetch()` call to your own tiny backend (even a 30-line Node.js script) that calls TikTok's API and returns orders as JSON.

Important: browsers block direct API calls to TikTok (CORS + secret keys must never live in frontend code). You need a small backend in the middle. Ask Claude Code: *"Create a server.js using Node + Express with one endpoint /api/orders that calls the TikTok Shop order search API with my app_key/app_secret and returns the orders as JSON. Then update index.html to fetch from /api/orders instead of using sample data."* Claude Code will scaffold this for you.

Agency shortcut: TikTok's app authorization is per-shop. Authorize once per client shop, store each shop's token, and your multi-shop tabs (Prompt 3) map 1:1 to real client stores. That's the whole agency model in one dashboard.

14. Make It Yours — Customization Ideas

Try these as follow-up prompts to Claude Code (one at a time):

- *"Add a dark/light mode toggle to the dashboard."*
- *"Add WhatsApp alerts: when an order goes at-risk, show a button that opens wa.me with a pre-filled reminder message."*
- *"Add a search box to find orders by customer name or order id."*

- "Add an Urdu language toggle — all labels switch between English and اردو."
- "Add a daily summary panel: orders dispatched per courier today."
- "Make the design match my brand colors: primary #_____, accent #_____."

15. Put It Online (Free Hosting)

1. **GitHub Pages (free):** create a free GitHub account → new repository → upload `index.html` → Settings → Pages → deploy. You get a public link in 2 minutes.
2. **Netlify Drop:** go to `app.netlify.com/drop` → drag your `index.html` → instant live link.
3. **Vercel:** similar drag-and-drop deploy, free tier.

For the API-connected version (Section 13), deploy the backend on **Render** or **Railway** free tier, and point your frontend `fetch()` at it.

16. Troubleshooting

Problem	Fix
Countdowns don't tick	Ask Claude Code: "the setInterval countdown isn't updating — debug it." Usually a variable name mismatch.
Blank white page	Press F12 → Console → read the red error → paste it into Claude Code and ask for a fix.
Claude Code changed something I liked	Say "revert the last change to [the stats bar]" — or keep backups: save a copy of <code>index.html</code> before each big prompt.
Prompt output is too long / gets cut	Ask it to "continue" — or split the prompt in half and send as two messages.
Urdu text shows as boxes	Add <code><meta charset="utf-8"></code> at the top of the HTML head (Claude Code usually includes it).

TikFulfill — concept by Imtiaz Ahmad, AI Soft Tech Solution (aisofttechsolution.com). This guide is free to share. Built as a demo of what's possible when small businesses get real software: the platform gives you the orders — handling them is your system's job.

Questions? Reach out: imtiazahmad@aisofttechsolution.com · +44 7462 086661